

The Brutal Q&A

Every hard question about Inayat, answered simply — for you, not for a slide

This covers every question you named, plus the harder ones a sharp judge asks next. Every answer is short on purpose. Read it twice, out loud, before you go up.

1. The problem — from zero, no background needed

Picture this first, before any numbers: Someone is sick. They travel to a government hospital and wait in a long line, sometimes for hours. When it's finally their turn, the doctor is exhausted, there are dozens more patients waiting outside, and the doctor has only a couple of minutes before the next patient has to come in. In that short time, the doctor has to figure out what's wrong, using nothing but what the patient manages to say in those few minutes. That's the whole problem, in one picture. Everything below is just proof that this picture is real, everywhere, every day — not a one-off bad experience.

Q: What is the actual problem, explained from the very start?

A: In a government hospital in India, a doctor sees a patient for about 2 minutes. That's it. In 2 minutes, a doctor cannot properly ask what's wrong, listen to the full story, AND read old prescriptions. Something has to be rushed — and it's almost always the listening part. But doctors say listening carefully is the single biggest reason they get a diagnosis right in the first place.

Q: Why does this actually matter for real people, not just in theory?

A: About two out of every three people in India live in villages. But only about 1 in 4 doctors work in villages. So the people with the least access to a doctor are also the ones getting the least time once they finally get one. That's the real gap.

Q: How did you solve it, in one sentence?

A: We built something that talks to the patient BEFORE the doctor does — typing, speaking, or a photo of an old prescription — and turns that into a clean, organized note. So by the time the doctor walks in, the listening is already done.

2. The pipeline — what happens, step by step, and why

- **Step 1 — Consent.** The patient has to agree before we save anything. Not a checkbox buried somewhere — the system refuses to save without it. Why: it's the patient's data, and that's simply the right thing to do.
- **Step 2 — The patient talks.** Type, speak in their own language, or show an old prescription photo. Why: not everyone can type well, and not everyone speaks English — so we don't force one way.
- **Step 3 — Smart follow-up questions.** A cough gets asked about phlegm and colour. A skin problem gets asked about spreading and new soaps or foods. Chest or body pain gets the standard 8-question method real doctors are trained on. Why: one generic question list for every problem would be lazy and wrong.
- **Step 4 — The danger check, first, always.** Before any AI touches the case, we check the patient's exact words against a list of dangerous symptoms — “chest pain,” “can't breathe,” and more. This needs zero AI and zero internet. Why: a dangerous case should never depend on an AI model having a good day.
- **Step 5 — If it's not obviously dangerous, the AI helps.** It reads the case and suggests how urgent it is. If the AI isn't available (no internet, no key), a safe backup kicks in instead of the app just failing.
- **Step 6 — A second, independent double-check.** We compare the patient's words against real medical guideline text. This check can only push the urgency UP, never down. Why: a wrong correction toward “less serious” is far more dangerous than being extra careful.
- **Step 7 — The doctor decides.** Everything is saved. The doctor opens it, reads it, can change anything, and only then is it final. The AI never diagnoses. It never prescribes.

3. The engineering questions — pipeline, size, where it lives

Q: How much space (GB) does this actually take, and where does it run?

A: The real app code itself is tiny — about 4.5 MB, smaller than one phone photo. The extra Python libraries it actually needs to run (web server, AI connection, text matching, image reading, encryption, offline speech) add up to roughly 290 MB, measured directly, not guessed. One heavier library (PyTorch, for an image-recognition feature that isn't wired into the live app yet) is deliberately left OUT of the deployed version to avoid nearly 5 GB of dead weight — we checked and excluded it on purpose. It's hosted on Render, a cloud server company, on their free tier — which honestly means it goes to sleep after 15 minutes of no visitors and takes 30–60 seconds to wake back up on the next visit. We say that plainly instead of hiding it.

Q: What parameters or choices did you actually make, and why those?

A: A few real examples: we match patient text against medical guidelines using simple word-matching (not a giant AI model) because our reference list is only a few dozen short passages — a big model would be slower for no real benefit at that size. We only trust the SINGLE best guideline match, not the top three — because early on, a weak third-place match once overruled a correct first-place match and would have wrongly escalated a normal knee-pain case. We caught that ourselves, tested it, and fixed it.

Q: What tech did you actually build this with?

A: Python for the server, plain HTML and JavaScript for the app screen (no heavy framework, so it loads fast even on a cheap phone), a small database to save cases, Claude (an AI model) for language understanding, and real government tools — Bhashini for Indian-language voice, and India's own health-ID system (ABDM) for patient identity.

4. Why we're different — and the ONE thing nobody else has

Q: How is this better than another team's health chatbot?

A: Most health chatbots are one AI model with a disclaimer at the bottom. Ours has a real safety net that doesn't depend on the AI at all, keeps working with zero internet, saves the case for the doctor to reopen later, and connects to real government health systems instead of pretending to.

Q: If you had to name the ONE thing you have that basically nobody else building this has — what is it?

A: A safety check that can only make things MORE careful, never less — built so that even if the AI is completely wrong, or completely offline, a dangerous symptom still gets caught. Most student projects trust the AI's answer directly. We built a system that never fully trusts it.

Q: Isn't this just ChatGPT with extra steps?

A: No. Open ChatGPT and describe symptoms: nothing double-checks its answer, nothing is saved for a doctor to reopen, and it stops working completely with no internet. We built three separate, real things ChatGPT doesn't have, not one clever prompt.

5. Grabbing attention — the pitch, and the prototype itself

Q: How will you grab people's attention when you present this?

A: We open with the real number, not a claim: a doctor gets about 2 minutes per patient — one of the lowest in the world, from a real study of 67 countries. That number alone makes people lean in, because everyone has sat in that waiting room.

Q: Once someone is actually looking at the app, what makes IT grab them?

A: The homepage doesn't wait for anyone to click anything. A real example symptom types itself out on screen and gets caught as an emergency, live, before the visitor has touched anything — proof, not a promise. The page also reveals its content smoothly as you scroll instead of just dumping everything on screen at once, so it feels alive, not like a static form.

6. Limitations — and proof we actually KNOW them, not guessed

The difference between a real limitation and a guess: we tested it and watched it fail, with our own eyes, before writing it down.

- **Old handwritten prescriptions read worse than clean typed ones.** Proof: we tested the exact same reading code on a blurry image and a sharp one — the blurry one turned real numbers like “13.0-17.0” into wrong numbers like “130-170.” We saw it happen.
 - **Our medical reference list isn't officially certified yet.** We wrote it ourselves as a real starting point, and we say so — we don't pretend it's an official government document.
 - **The voice and health-ID features have never touched the real government servers.** No test credentials exist for us to try them live — only their public documentation. We built and tested everything we could without lying about the last step.
 - **Offline voice recognition (used only when there's no internet) is genuinely not very accurate yet.** We tested it ourselves and measured it — it's usable, but every result from it is automatically flagged “needs a human to double check,” never silently trusted.
-

7. The extra brutal round — the harder questions

Q: What's the hardest real bug you personally found and fixed?

A: Our safety-check used to look at the top 3 closest guideline matches and trust the most serious one among them. We found, by testing it ourselves, that a weak, barely-related third match could override a correct, strong first match — it once wrongly turned ordinary “mild knee pain” into an EMERGENCY. We caught it, proved it with a test, and fixed it to only ever trust the single best match. That's a real bug we found before anyone else did, in a safety-critical part of the system.

Q: What if the AI gives a wrong answer during the actual demo?

A: That's exactly why the danger-word check runs first with zero AI, and why the second guideline check can only make things MORE careful. A wrong AI answer alone still can't produce an unsafe result — it would take two independent systems being wrong in the same direction at once.

Q: Couldn't a big company just build this in a week and crush you?

A: A big company could build the AI-chatbot part in a week, sure. What takes real, deliberate work is the safety-net design, the real government integrations, and actually testing against messy real-world input — none of that is a weekend job, and most teams skip it entirely.

Q: Is patient data actually safe?

A: The doctor's screen sits behind a real login. Without the correct passcode, the server itself refuses every request for patient data — it's not just a hidden button someone could work around.

Q: Have you tested this on real patients in a real hospital?

A: No, and we say that honestly rather than imply otherwise. We've tested it thoroughly with 358 automated tests and real example cases we wrote ourselves. Real-patient testing is a genuine next step, not something we're claiming already happened.

Q: If one of you gives a different answer than another teammate on stage, does that look bad?

A: No — it looks bad only if someone guesses. If your answers differ slightly, that's fine. If you don't know something, say so and offer to follow up. That's what this whole document is for — so you don't have to guess.

8. The answers you say without thinking

The whole thing, in one line: “Inayat listens to a patient before the doctor does, and turns what they say into a safe, organized note — so the doctor's few minutes go to deciding, not repeating questions.”

Three reasons people should respect this project:

- We tell you what's NOT finished, out loud, before you can catch us — that's rarer than it should be.
- The safety design doesn't trust one single thing to be right — it's built to catch itself being wrong.
- Every real government feature we claim, we actually built and tested — nothing here is a slide with no code behind it.

You've now read the real problem, the real pipeline, the real numbers, the real bugs we found ourselves, and the real limits — in your own words. That's not luck. That's understanding. Go say it.